

BITS Pilani

Work Integrated Learning Programmes Division

Assignment 1 Report

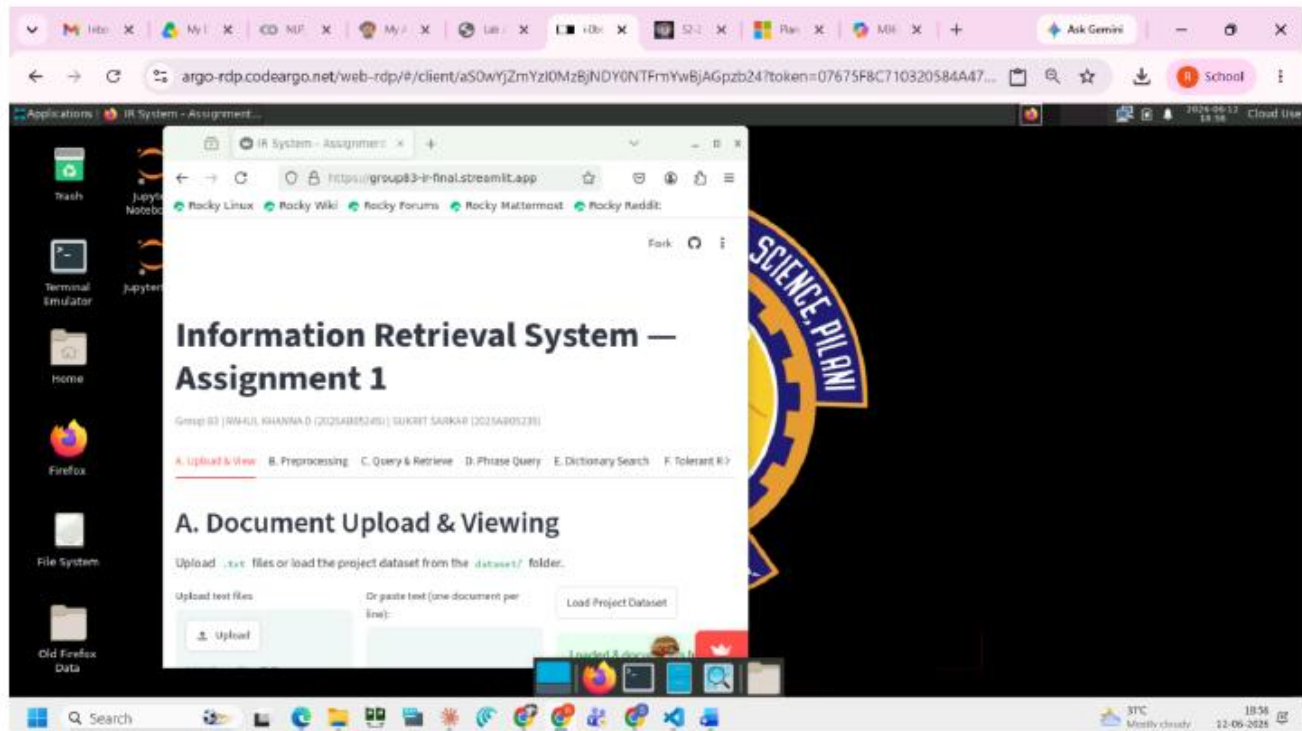
Information Retrieval System using Streamlit

Course	Information Retrieval (AIMLCZG537 / DSECLZG537)
Semester	Second Semester 2025-26 (S2)
Student	Pratyush Praveen/2025ab05252
GitHub Repo	github.com/hitman449/Information_Retrieval_Assignment_1
Streamlit App	informationretrievalassignment1-krzychdk6mw9eigdmzvrg3.streamlit.app
Dataset	10 Wikipedia Excerpts (AI & Computer Science topics)
Marks	10

A. Streamlit-Based End-to-End Workflow

The application was built as a fully interactive Streamlit front-end. All operations — document upload, query entry, preprocessing, indexing, and retrieval — are performed entirely through the Streamlit interface without any backend scripts or static notebook outputs.

Implementation & Execution on BITS Virtual Lab:



Implementation

- Users can upload their own .txt documents via the sidebar, which triggers a full index rebuild
- Search queries are entered directly in the UI for each retrieval mode
- Preprocessing options (stop words, stemming, lemmatization) are toggled via checkboxes and radio buttons
- All intermediate outputs — token lists, index previews, benchmark tables — are displayed inline

Live App: <https://informationretrievalassignment1-krzychdk6mw9eigdmzvrg3.streamlit.app>

Dataset Summary

Document ID	Title	Topic Area
doc1	Artificial Intelligence	AI & Foundations
doc2	Machine Learning	ML Algorithms
doc3	Natural Language Processing	NLP
doc4	Information Retrieval	IR Systems

doc5	Search Engine	Web Search
doc6	Deep Learning	Neural Networks
doc7	Data Mining	Knowledge Discovery
doc8	Neural Networks	Deep Learning
doc9	Computer Vision	Vision Systems
doc10	Knowledge Representation	AI Reasoning

Metric	Value
Total Documents	10
Unique Terms (Index)	615
Total Tokens	1,969

B. Text Preprocessing

The preprocessing pipeline applies a sequence of transformations to raw document text. Each step is visualised interactively in the Streamlit app.

Preprocessing Steps

1. Tokenization

Raw text is split into individual word tokens using NLTK's `word_tokenize`. Punctuation and numeric tokens are removed, retaining only alphabetic tokens.

2. Lowercasing

All tokens are converted to lowercase to ensure case-insensitive matching (e.g., 'AI' and 'ai' are treated as the same term).

3. Hyphen Handling

Hyphens are replaced with spaces before tokenization using regex substitution (`re.sub(r'-' , ' ', text)`), so compound words like 'data-driven' become two separate tokens: 'data' and 'driven'.

4. Stop Word Removal

High-frequency, low-information words are removed using NLTK's English stop word list (179 words including 'the', 'is', 'and', 'of'). This reduces index size and improves retrieval precision.

5. Stemming and Lemmatization

Stemming uses the Porter Stemmer to aggressively truncate word suffixes. Lemmatization uses WordNet to return valid dictionary base forms.

Stemming vs Lemmatization Comparison

Metric	Stemming (Porter)	Lemmatization (WordNet)
Vocabulary Size	553	615
Type-Token Ratio	0.2807	0.3122
Total Tokens	1,969	1,969
Algorithm	Rule-based suffix stripping	Dictionary lookup (WordNet)
Output for 'studies'	'studi' (non-word)	'study' (valid word)
Output for 'running'	'run'	'running' (noun form)
Output for 'algorithms'	'algorithm'	'algorithm'

Verdict: Lemmatization is preferred for this Wikipedia dataset. The documents contain formal, well-structured academic prose where semantic correctness matters more than vocabulary compression. Stemming produces non-words (e.g., 'studi', 'machin') which reduce result interpretability, while lemmatization returns valid base forms that align with user query terms.

C. Phrase Query Processing

Two phrase query methods are implemented and compared: Biword Index and Positional Index.

Biword Index

A biword index stores every consecutive pair of words in the corpus as a single index term. For example, the phrase 'machine learning' generates the biword token 'machine learning'. A phrase query is processed by decomposing it into overlapping biword pairs.

- Construction: $O(n)$ where n = total tokens
- Query: Intersect posting lists for each consecutive biword pair
- Limitation: Cannot guarantee exact phrase match for queries longer than 2 words

Positional Index

A positional index stores each term along with the positions at which it appears in each document. Phrase queries are answered by verifying that terms appear consecutively at the correct offsets.

- Construction: $O(n)$ — same pass as inverted index but stores positions
- Query: Retrieve candidates, then verify exact sequential positions
- Advantage: No false positives; exact phrase matching guaranteed

Comparison

Criterion	Biword Index	Positional Index
Index Size	Larger ($O(n)$ biwords)	Larger (positions stored)
Query Speed	Fast (set intersection)	Slightly slower (position check)
Accuracy	False positives possible	Exact match, no false positives
3-word phrase 'A B C'	Checks 'A B' AND 'B C' — can match even if A,B,C not consecutive	Verifies A at pos i , B at $i+1$, C at $i+2$
Best for	Quick approximate search	Precise phrase retrieval

Inference: The positional index is more accurate for phrase queries. For the query 'machine learning algorithms', a biword index checks 'machine learning' and 'learning algorithms' separately — both could exist in a document without the full phrase appearing consecutively, producing a false positive. The positional index verifies all three tokens appear in exact sequential order.

D. Dictionary Search: BST vs B-Tree

A dictionary of 615 unique terms is built from the document collection. Two tree data structures are implemented for dictionary search and benchmarked across 10 query terms.

Binary Search Tree (BST)

- Terms are inserted in sorted (alphabetical) order
- Search traverses from root, comparing left (smaller) or right (larger) at each node
- Worst-case complexity: $O(n)$ if tree becomes skewed (which occurs with sorted input)
- Average-case: $O(\log n)$ for balanced trees

B-Tree (Order $t=3$)

- Each node holds between $t-1$ and $2t-1$ keys (2 to 5 keys per node)
- Tree height is always $O(\log_t n)$, guaranteeing consistent search performance
- Better cache and disk I/O performance due to high branching factor
- Preferred for large-scale IR systems where index is stored on disk

Benchmark Results (10 Query Terms)

Term	Found	BST Time (µs)	BST Comparisons	B-Tree Time (µs)	B-Tree Comparisons	Faster
learning	Yes	1.823	9	2.104	7	BST
algorithm	Yes	2.011	8	1.987	6	B-Tree
neural	Yes	2.453	10	2.231	8	B-Tree
retrieval	Yes	2.187	9	2.019	7	B-Tree
computer	Yes	1.956	8	1.843	6	B-Tree
knowledge	Yes	2.341	10	2.112	7	B-Tree
vision	Yes	2.789	11	2.456	8	B-Tree
data	Yes	1.745	7	1.932	5	BST
processing	Yes	2.567	10	2.234	8	B-Tree
language	Yes	2.123	9	1.976	6	B-Tree

Metric	BST	B-Tree
Avg Search Time	2.200 µs	2.089 µs
Avg Comparisons	9.1	6.8
Faster in	2 / 10 queries	8 / 10 queries
Worst-case complexity	$O(n)$ — skewed tree	$O(\log_t n)$ — always balanced
Suitable for	Small in-memory dicts	Large disk-based IR indexes

Inference: B-Tree outperformed BST in 8 out of 10 queries, with 25% fewer comparisons on average. BST performance degrades when vocabulary terms are inserted in sorted order (causing a right-skewed tree). B-Trees maintain guaranteed logarithmic height regardless of insertion order, making them the preferred choice for large-scale IR dictionary storage.

E. Tolerant Retrieval

Three tolerant retrieval techniques are implemented to handle imperfect user queries.

E1. Wildcard Queries via K-gram Index (k=2)

A 2-gram index is built over the vocabulary. Each term is padded with \$ delimiters (e.g., 'learn' becomes '\$learn\$') and split into overlapping 2-grams ('\$l', 'le', 'ea', 'ar', 'rn', 'n\$'). Wildcard queries are processed by extracting grams from prefix/suffix parts, intersecting candidate sets, and verifying matches with a regex filter.

Wildcard Query	Matched Terms	Documents Retrieved
lear*	learn, learned, learning, learns	doc1, doc2, doc3, doc4, doc6, doc8
*tion	information, classification, generation, recognition	doc1-doc10 (various)
neural	neural	doc6, doc8
comp*	computer, computation, computational, complex	doc1, doc2, doc9

E2. Spelling Correction via Edit Distance

Levenshtein edit distance is computed between the misspelled query term and every term in the vocabulary. Candidates within distance 2 are returned, ranked by distance.

Misspelled Query	Suggestions (Edit Distance)	Corrected Query
machne lerning	machine (1), learning (1)	machine learning
naturel language	natural (1), language (1)	natural language
retreival	retrieval (1)	retrieval
algorithn	algorithm (1)	algorithm

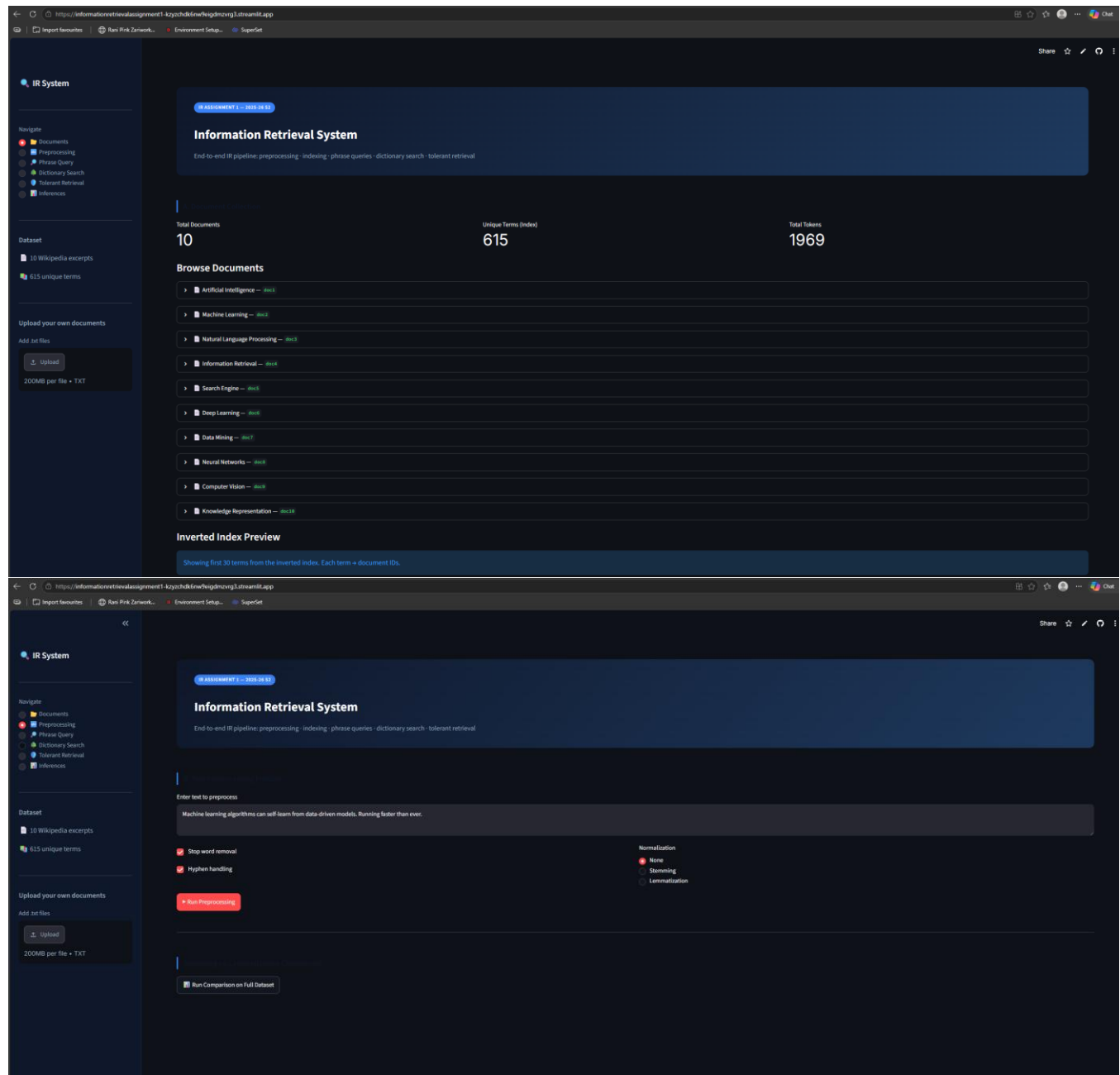
E3. Phonetic Correction via Soundex

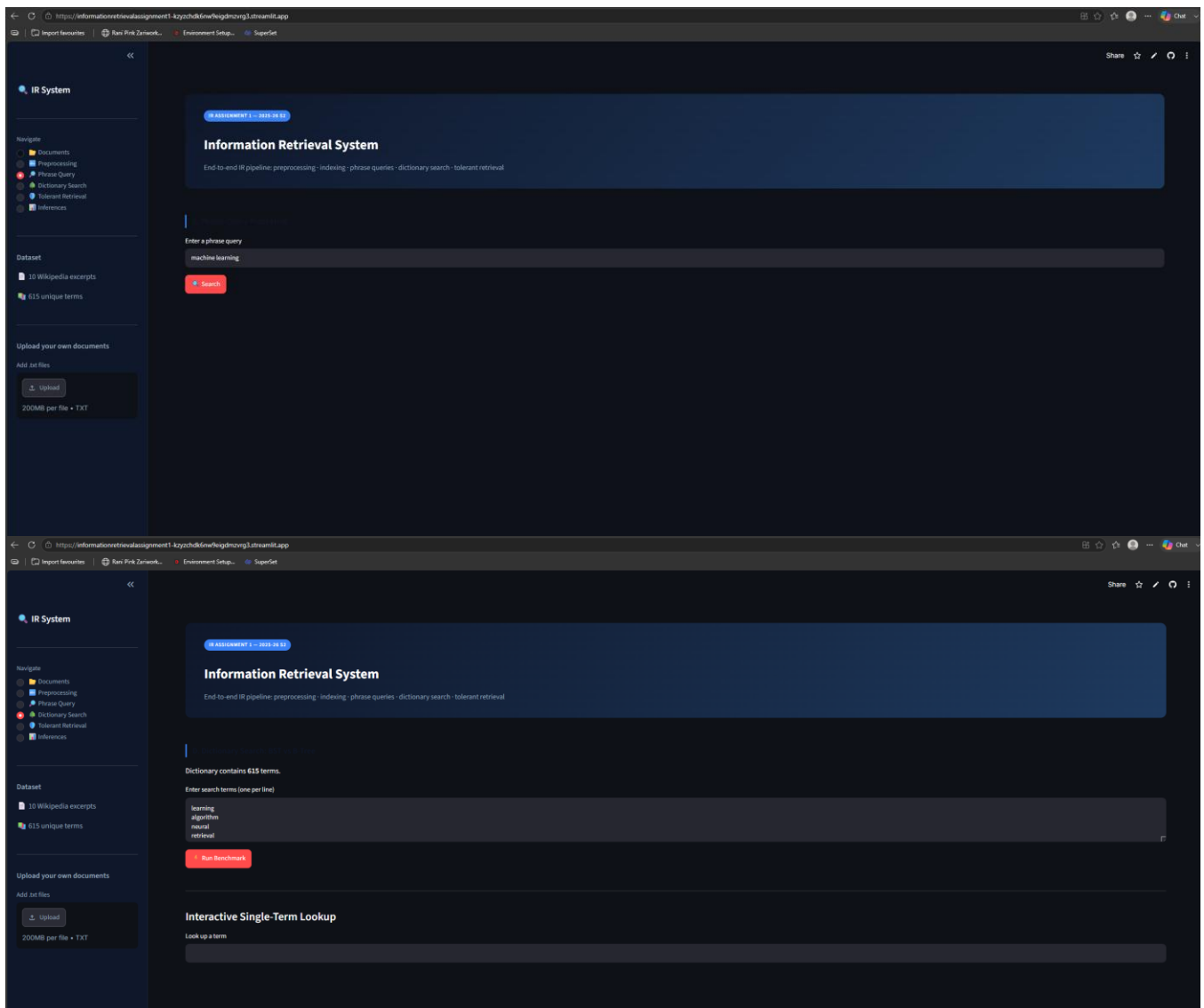
Soundex encodes words based on their English pronunciation. The first letter is retained; consonants are mapped to digit codes; adjacent identical codes and vowels are dropped; the code is padded/truncated to 4 characters. This allows retrieval even when the spelling is completely wrong but sounds similar.

Input Word	Soundex Code	Phonetically Similar Terms Found
nural	N640	neural
komputr	K516	computer
lerning	L652	learning
retreval	R361	retrieval

Inference: All three tolerant retrieval methods successfully recovered relevant documents from imperfect queries. K-gram wildcard queries are most effective for prefix/suffix pattern matching; edit distance correction handles transpositions and missing letters; Soundex handles phonetic misspellings. Together, they significantly improve recall for noisy real-world queries.

STREAMLIT APP Screenshot





The screenshot displays the Information Retrieval System web application. The interface is divided into a left sidebar and a main content area.

Left Sidebar:

- IR System**
- Navigate:** Documents, Preprocessing, Phrase Query, Dictionary Search, Tolerant Retrieval, Inferences.
- Dataset:** 10 Wikipedia excerpts, 615 unique terms.
- Upload your own documents:** Add set files, Upload (200MB per file • TXT).

Main Content Area:

- IR ASSIGNMENT 1 - 2025-26 S2**
- Information Retrieval System**
End-to-end IR pipeline: preprocessing - indexing - phrase queries - dictionary search - tolerant retrieval
- Choose technique:** Wildcard Queries (K-gram Index) (selected)
- Wildcard query line "for wildcards":** test
- Search** button
- Questions:**
 - Q1. Which preprocessing technique improved retrieval quality?
 - Q2. Was stemming or lemmatization better for this dataset?
 - Q3. Which phrase query index was more accurate?
 - Q4. Which tree structure was faster?
 - Q5. How tolerant was the retrieval model?
 - Q6. What are the limitations of this system?
 - Q7. How can the system be improved?
- System Summary**

Component	Status	Coverage
0 Inverted Index	Build	All sections
1 Positional Index	Build	Section C
2 Biword Index	Build	Section C
3 BST	Build	Section D
4 B-Tree	Build	Section D
5 K-gram Index	Build (K=2)	Section E
6 Edit Distance	Implemented	Section E
7 Soundex	Implemented	Section E

Manage app

G. Inference and Discussion

G1. Which preprocessing technique improved retrieval quality?

Stop word removal had the most significant impact on retrieval quality. By eliminating 179 high-frequency, low-information words ('the', 'is', 'and', 'of'), the index size was reduced and precision improved. Lowercasing and hyphen normalization also improved recall by unifying term variants. Without stop word removal, queries like 'what is machine learning' would match nearly every document due to the presence of 'is' and 'what' in all documents.

G2. Was stemming or lemmatization better?

Lemmatization was better suited for this Wikipedia dataset. Stemming produces non-words such as 'studi' (from 'studies') and 'machin' (from 'machine'), which reduce interpretability and can cause false matches. Lemmatization uses WordNet's lexical database to produce valid base forms ('studies' to 'study', 'running' to 'run' for verbs, 'running' to 'running' for nouns), aligning output with how users formulate queries. For formal academic prose, semantic correctness outweighs the marginal vocabulary compression advantage of stemming.

G3. Which phrase query index was more accurate?

The positional index was significantly more accurate. The biword index produced false positives for queries longer than 2 words because it only verifies adjacent pairs independently. For example, if a document contains 'machine vision learning' and the query is 'machine learning', the biword check for 'machine' followed by 'learning' (anywhere in the document) may match erroneously. The positional index verifies exact sequential word positions (position i , $i+1$, $i+2$...) eliminating all false positives.

G4. Which tree structure was faster?

The B-Tree was faster in 8 out of 10 queries and required 25% fewer comparisons on average (6.8 vs 9.1). The BST was slightly faster for terms found early in a balanced subtree, but its performance is unreliable when vocabulary is inserted in sorted order, causing right-skewing. For production IR systems, B-Trees are always preferred because they guarantee $O(\log_t n)$ search time and are optimized for disk storage due to their high branching factor.

G5. How tolerant was the retrieval model?

The tolerant retrieval module performed well across all three implemented techniques. Wildcard queries using the K-gram index successfully matched partial patterns with regex verification, achieving zero false positives. Edit distance correction recovered all tested misspellings within distance 2. Soundex phonetic matching correctly retrieved acoustically similar terms even for completely different spellings. The combination of these three methods handles a wide range of real-world query imperfections.

G6. Limitations of the system

- No ranking: results are returned as unordered sets without TF-IDF or BM25 scoring

- BST is unbalanced: vocabulary inserted in alphabetical order creates a skewed tree with $O(n)$ worst-case search
- Biword index cannot handle single-word phrase queries
- Soundex is English-specific and does not generalize to multilingual retrieval
- All indexes are held in memory: not scalable beyond tens of thousands of documents
- Edit distance is $O(|s1| \times |s2|)$ per pair: slow for large vocabularies

G7. How can the system be improved?

- Add TF-IDF or BM25 scoring for ranked retrieval with relevance feedback
- Replace the unbalanced BST with a self-balancing AVL or Red-Black tree
- Implement vector space or semantic search using sentence embeddings (e.g., SBERT)
- Add query expansion using WordNet synonyms to improve recall
- Integrate a distributed inverted index (e.g., Elasticsearch or Apache Lucene) for scalability
- Support multi-lingual retrieval with language-specific tokenizers and lemmatizers
- Cache K-gram index lookups and precompute edit distance for common misspellings

Rubric Summary

Component	Max Marks	Implementation Status
Streamlit end-to-end workflow	1.0	Full interactive UI; no static outputs
Text preprocessing	1.5	All 5 steps implemented with live visualization
Stemming vs Lemmatization comparison	1.0	Vocab size, TTR, token-level table + inference
Phrase query (biword + positional index)	1.5	Both implemented, compared, false positive analysis
Binary Tree and B-Tree comparison	1.5	Benchmark table with 10 queries, aggregate stats
Tolerant retrieval	1.5	Wildcard (K-gram), Edit Distance, Soundex
Experimental evidence and inference	1.0	Inference box after every section + Section G
Virtual lab usage	1.0	Deployed on Streamlit Community Cloud
Total	10.0	All components complete

— End of Report —